AI

TalentScout

AI Hiring Assistant

An AI-powered chatbot for automated candidate screening
using Large Language Models, NLP & Prompt Engineering

Project	TalentScout AI Hiring Assistant
Domain	Artificial Intelligence / Natural Language Processing
Technology	Python | Streamlit | Groq API (Llama 3) | scikit-learn
Type	Internship / Academic Project Submission

1. Introduction

The **TalentScout AI Hiring Assistant** is an AI-powered conversational chatbot designed to automate the initial screening stage of the recruitment process. Traditional hiring workflows require recruiters to manually collect candidate information, formulate technical questions, and evaluate responses — a process that is both time-consuming and prone to human inconsistency.

This project addresses these challenges by leveraging **Large Language Models (LLMs)** to streamline candidate screening end-to-end. The system interacts with candidates through a conversational interface, gathers structured information, generates technical questions based on their declared technology stack, evaluates responses, and delivers structured feedback along with a final hiring decision.

The result is a scalable, intelligent, and consistent screening system that significantly reduces recruiter effort while maintaining high evaluation quality and objectivity.

2. Objectives

The primary objectives of this project are:

- Automate the initial candidate screening process, reducing manual recruiter effort.
- Dynamically generate technical interview questions based on each candidate's declared technology stack.
- Evaluate candidate responses using AI, providing consistent and objective scoring.
- Deliver structured feedback including a numerical score, sentiment classification, and written explanation.
- Detect response originality using NLP techniques to identify generic or copied answers.
- Produce a final screening verdict (**PASS / FAIL**) based on aggregate performance.

3. System Architecture

The system follows a modular, layered architecture integrating the user interface, a prompt engineering layer, an LLM engine, and a downstream evaluation pipeline.

3.1 Data Flow Overview

Layer	Component	Responsibility

Input	User (Candidate)	Submits responses via chat interface
Presentation	Streamlit UI	Renders conversation, sidebar profile, and progress
Intelligence	Prompt Engineering Layer	Structures inputs into precise LLM prompts
Processing	Groq API — Llama 3	Generates questions and evaluates answers
Analysis	Evaluation Engine	Extracts score, sentiment, and feedback via regex
Output	Decision Engine	Computes final PASS/FAIL based on average score

4. Features

4.1 Candidate Information Collection

The chatbot begins by collecting essential candidate details to personalise the entire interview session:

- Full Name
- Email Address
- Phone Number
- Years of Experience
- Desired Position
- Current Location
- Tech Stack (declared technologies)

4.2 Tech Stack-Based Question Generation

Technical questions are dynamically generated based on the candidate's declared technology stack. This ensures questions are contextually relevant and appropriately challenging. For example, a candidate declaring **Python, Java, and C++** will receive tailored questions for each technology, simulating a real technical interview.

4.3 Answer Evaluation

Each candidate response is evaluated across three dimensions, producing three outputs:

Evaluation Dimension	Description
Score (0–10)	Numerical rating of technical accuracy and completeness
Sentiment	Classified as Confident, Neutral, or Uncertain
Feedback	A 2–3 sentence written explanation from the LLM

4.4 Sentiment Analysis

The system classifies the confidence level embedded in each candidate response:

Sentiment	Interpretation
Confident	Clear articulation and strong technical understanding demonstrated
Neutral	Partial understanding with some gaps or ambiguity
Uncertain	Weak, vague, or technically incorrect response provided

4.5 Originality Detection

To flag generic or copied responses, the system applies **TF-IDF vectorisation** and **cosine similarity** against a reference corpus of common answers. The result is an **Originality Score (%)** that quantifies how unique the candidate's response is.

Example output: Originality Score: 92% — indicating a highly original, candidate-authored response.

4.6 Final Screening Decision

Upon completion of all questions, the system computes a final hiring verdict:

Metric Used	Average score across all evaluated responses
PASS Condition	Average score ≥ 6.0
FAIL Condition	Average score < 6.0
Output	Final verdict with a personalised candidate summary message

5. Technologies Used

Category	Technology / Tool
Programming Language	Python 3.x
UI Framework	Streamlit
LLM Provider	Groq API — Llama 3 Model
NLP / Similarity	scikit-learn (TF-IDF, Cosine Similarity)
Environment Management	python-dotenv
Text Parsing	Python re (Regular Expressions)
Deployment Platform	Streamlit Cloud
Secrets Management	Streamlit Secrets / .env files

6. Implementation Details

6.1 Frontend — Streamlit

The user interface is built entirely with Streamlit, providing:

- A chat-based interaction panel simulating a live interview conversation.
- A sidebar displaying the candidate profile, current question, and progress tracker.
- Responsive layout suitable for both desktop and mobile devices.
- Real-time state persistence using Streamlit's st.session_state API.

6.2 Prompt Engineering

Carefully structured prompts form the core intelligence of the system. Each prompt includes explicit output format instructions to produce consistent, machine-readable responses:

Prompt Task	Design Approach
Question Generation	Role context, tech stack, and difficulty level embedded in prompt
Answer Evaluation	Output format specified: Score / Sentiment / Feedback
Sentiment Detection	LLM classifies confidence from linguistic patterns in the response

6.3 Evaluation Engine

Each LLM response passes through the evaluation pipeline:

- Custom regex patterns extract Score (0–10), Sentiment label, and Feedback text.
- Results are stored in session state for persistence across conversation turns.
- A running average score is updated after each response for final decision computation.

6.4 Originality Module

- Candidate responses are vectorised using **TF-IDF**.
- Cosine similarity is computed against a predefined reference corpus.
- Originality Score is derived as: **(1 – similarity) × 100%**.

6.5 Deployment

- Hosted on **Streamlit Cloud** — publicly accessible via web URL.
- API keys secured via Streamlit Secrets in production; .env files for local development.
- No infrastructure management required — fully serverless deployment.

---

7. Challenges Faced

The following technical challenges were encountered during development:

Challenge	Detail
API Error Handling	Managing Groq API rate limits and handling unexpected failures gracefully
Conversation State	Maintaining multi-turn context within Streamlit's stateless execution model
Prompt Consistency	Designing prompts that reliably produce structured, parseable LLM output
Response Parsing	Extracting structured data (score, sentiment) from free-form LLM text
API Key Security	Preventing accidental exposure of credentials in source code or repositories

---

8. Solutions Implemented

Solution	Implementation
Structured Prompting	Explicit output format instructions in every prompt to control LLM behaviour
Regex Parsing	Custom patterns for reliable extraction of score, sentiment, and feedback
Session State	st.session_state used to persist conversation history and evaluation data
Secret Management	API keys in .env locally; Streamlit Secrets in production environment
Fallback Handling	Default values and error handlers prevent crashes from unexpected responses

9. Future Enhancements

The following enhancements are planned for subsequent development iterations:

- **Multilingual Support** — Enable candidate interactions in multiple languages for global reach.
- **Resume Parsing** — Automatically extract skills and experience from uploaded CV documents.
- **Adaptive Questioning** — Dynamically adjust question difficulty based on real-time candidate performance.
- **Analytics Dashboard** — Provide recruiters with visual insights on candidate cohort performance.
- **ATS Integration** — Connect with Applicant Tracking Systems (e.g. Greenhouse, Workday).
- **Voice Interface** — Support spoken responses for a more natural, accessible interview experience.

10. Conclusion

The **TalentScout AI Hiring Assistant** demonstrates how modern AI and Large Language Models can be applied to solve real-world recruitment challenges. The system successfully automates the initial screening workflow, reducing manual effort while delivering consistent, intelligent, and scalable candidate evaluations.

By integrating prompt engineering, NLP-based originality detection, and cloud deployment, this project provides a comprehensive example of practical AI application in human resources technology. The modular architecture ensures that individual components can be independently enhanced or replaced as requirements evolve.

This project highlights both the technical depth achievable with LLM-powered systems and the importance of thoughtful design in building AI applications that are reliable, fair, and production-ready.

11. References

- Groq API Documentation — https://console.groq.com/docs
- Streamlit Documentation — https://docs.streamlit.io
- Meta AI — Llama 3 Model — https://ai.meta.com/llama
- scikit-learn Documentation — https://scikit-learn.org
- Python Documentation — https://docs.python.org/3